

Assignment 1 – Traffic Light

Description:

Create a Traffic Light, using one Red, one Yellow, and one Green LED. The Green LED should be on for 6 seconds, then the light should switch to the Yellow light which should be on for 1.5 seconds, then switch to Red for 5 seconds. Then repeat the cycle 2 more times. The traffic light is controlled with a raspberry pi GPIO pins which are programmed in two different ways. One way is using the pigpio library and the other is using sysfs to control the behavior of the gpio pins of the pi.

Approach / What I Did:

I first began the assignment by building the connections on the breadboard. I originally thought that I would have to make a complex circuit like the one used for assignment 0 since this also involves blinking multiple LEDs but I figured that was out of the scope of the assignment so I opted to have each LED connected to separate GPIO pins. I made sure that between the LED and the power was a resistor as not to damage any components. I noticed that the green LED required more resistance to get its brightness to the same level as the other LEDs so the yellow and red LEDs both have 220 ohm resistors connected to them whereas the green LED has a 1K ohm resistor connected to it.

Once I had the circuit finished I began coding. The first library that we had to use was the PiGPIO library which was pretty easy to understand how to use with the documentation provided. With this library, I simply called the functions from it and only had to make a single wrapper function to make an LED turn on and off within a given time interval.

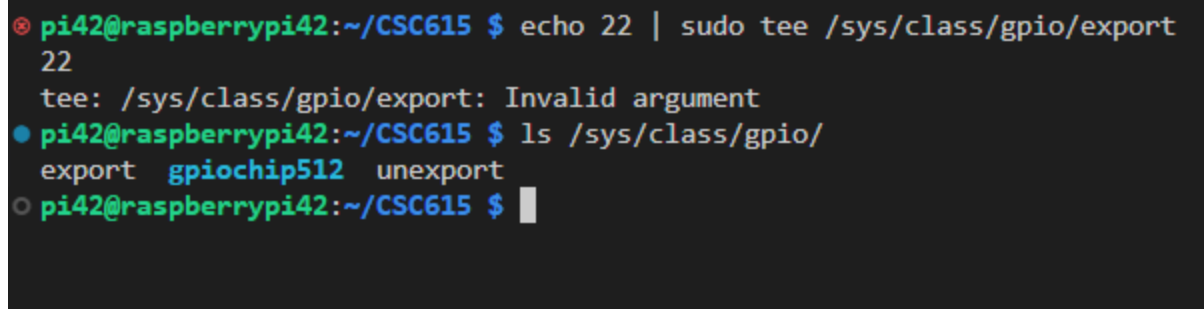
As for the second version of the code, I decided to use I/O (sysfs) operations to make the traffic light. I went with this approach as the documentation for direct register access seemed more verbose and I did not want to mess something up in the raspberry pi by mistake since direct register access is more low level.

Through programming the traffic light with I/O operations, I got a refresher on opening and writing to files in the system by error checking and being careful about buffer overflows. I made the basic functions for exporting, unexporting, setting direction, and setting values for the gpio pins. Then I made wrapper functions that used these functions in the context of controlling LEDs so that the main code is more readable.

After this, I made the makefile. In retrospect, I believe this should have been the first step as it would have saved me a lot of time in manually compiling and deleting the executables when testing my code. The makefile compiles and removes both versions of the code at the same time but they can be run individually using make run1 and make run2 for using PiGPIO and sysfs respectively.

Issues and Resolutions:

One of the biggest issues I had was using sysfs to control the GPIO pins. When attempting to export a gpio pin, writing to the `/sys/class/gpio/export` would produce errors. Even trying to write to it from the terminal would not work.

A terminal window on a Raspberry Pi. The prompt is `pi42@raspberrypi42:~/CSC615`. The user enters `echo 22 | sudo tee /sys/class/gpio/export`. The output is `tee: /sys/class/gpio/export: Invalid argument`. The user then enters `ls /sys/class/gpio/`. The output is `export gpiochip512 unexport`. The prompt returns to `pi42@raspberrypi42:~/CSC615`.

```
pi42@raspberrypi42:~/CSC615 $ echo 22 | sudo tee /sys/class/gpio/export
22
tee: /sys/class/gpio/export: Invalid argument
pi42@raspberrypi42:~/CSC615 $ ls /sys/class/gpio/
export  gpiochip512  unexport
pi42@raspberrypi42:~/CSC615 $
```

As the above screenshot shows, writing GPIO pin 22 into the export file would not produce a new file for `gpio22` where I could then control its behavior. After looking online, I found a file that could be checked to see if those pins were already in use and maybe that was causing issues so I ran the following command that showed me the pins that were in use.

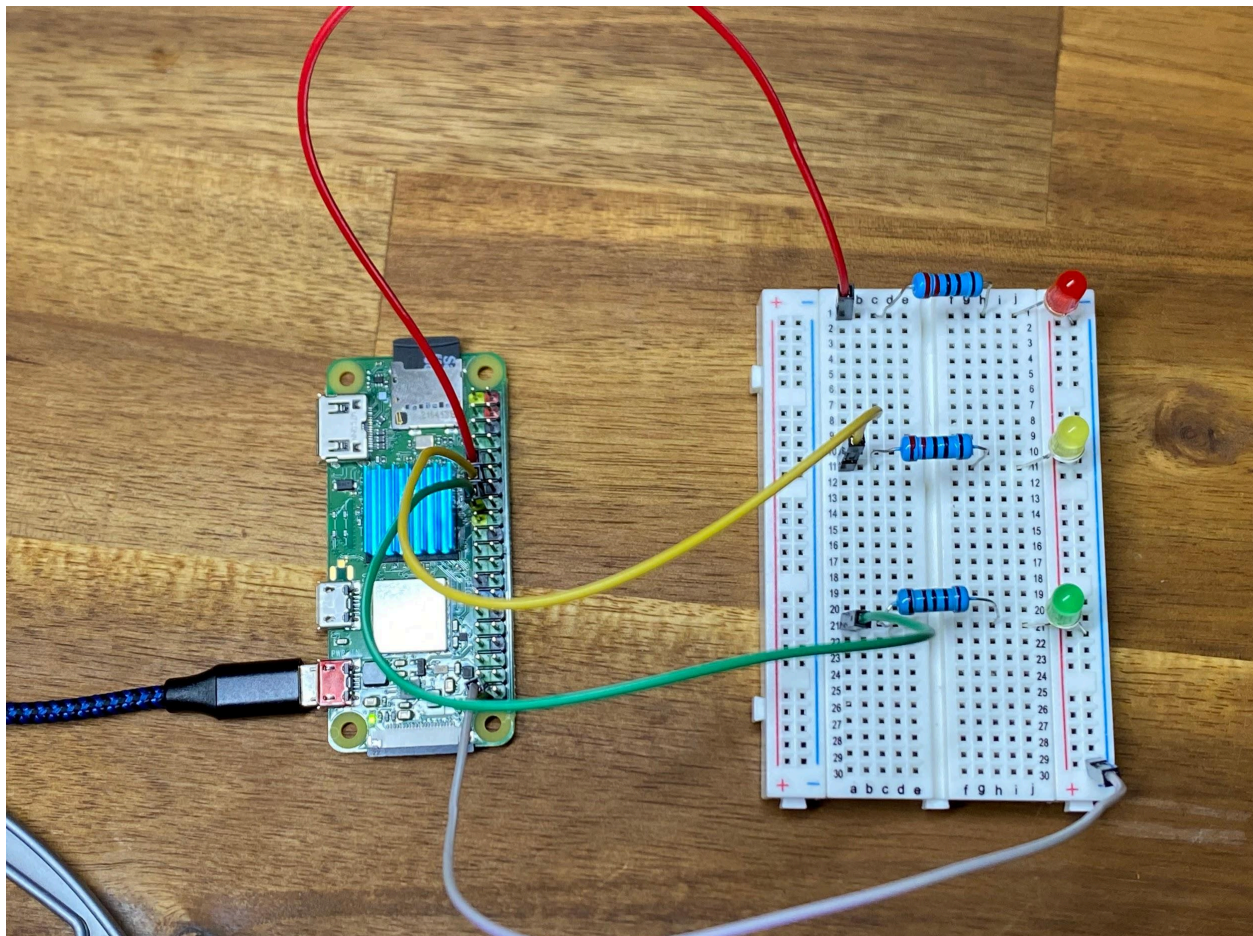
```
pi42@raspberrypi42:~/CSC615 $ cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 512-565, parent: platform/3f200000.gpio, pinctrl-bcm2835:
gpio-512 (ID_SDA) )
gpio-513 (ID_SCL) )
gpio-514 (GPIO2) )
gpio-515 (GPIO3) )
gpio-516 (GPIO4) )
gpio-517 (GPIO5) )
gpio-518 (GPIO6) )
gpio-519 (GPIO7) )
gpio-520 (GPIO8) )
gpio-521 (GPIO9) )
gpio-522 (GPIO10) )
gpio-523 (GPIO11) )
gpio-524 (GPIO12) )
gpio-525 (GPIO13) )
gpio-526 (GPIO14) )
gpio-527 (GPIO15) )
gpio-528 (GPIO16) )
gpio-529 (GPIO17) )
gpio-530 (GPIO18) )
gpio-531 (GPIO19) )
gpio-532 (GPIO20) )
gpio-533 (GPIO21) )
gpio-534 (GPIO22) )
gpio-535 (GPIO23) )
gpio-536 (GPIO24) )
gpio-537 (GPIO25) )
gpio-538 (GPIO26) )
gpio-539 (GPIO27) )
gpio-540 (HDMI_HPD_N) |hpd ) in hi ACTIVE LOW
gpio-541 (STATUS_LED_N) |ACT ) out lo ACTIVE LOW
gpio-542 (CTS0) )
gpio-543 (RTS0) )
gpio-544 (TXD0) )
gpio-545 (RXD0) )
gpio-546 (SD1_CLK) )
gpio-547 (SD1_CMD) )
gpio-548 (SD1_DATA0) )
gpio-549 (SD1_DATA1) )
gpio-550 (SD1_DATA2) )
gpio-551 (SD1_DATA3) )
gpio-552 (CAM_GPIO1) |cam1_regulator ) out lo
gpio-553 (WL_ON) )
gpio-554 (BT_ON) |shutdown ) out hi
gpio-555 (WIFI_CLK) )
gpio-556 (SDA0) )
gpio-557 (SCL0) )
gpio-558 (SMPS_SCL) )
gpio-559 (SMPS_SDA) )
gpio-560 (SD_CLK_R) )
gpio-561 (SD_CMD_R) )
gpio-562 (SD_DATA0_R) )
gpio-563 (SD_DATA1_R) )
gpio-564 (SD_DATA2_R) )
gpio-565 (SD_DATA3_R) )
pi42@raspberrypi42:~/CSC615 $
```

The pins that I wanted to use were not in use but I noticed that all GPIO pins were being referred to as GPIO<512 + GPIOpinNumber>. So to refer to the pin that I wanted to control I would have to write the GPIO pin number + 512. So if I wanted to use GPIO pin 22 I would need to refer to it as GPIO pin 534 when accessing the pin files using sysfs.

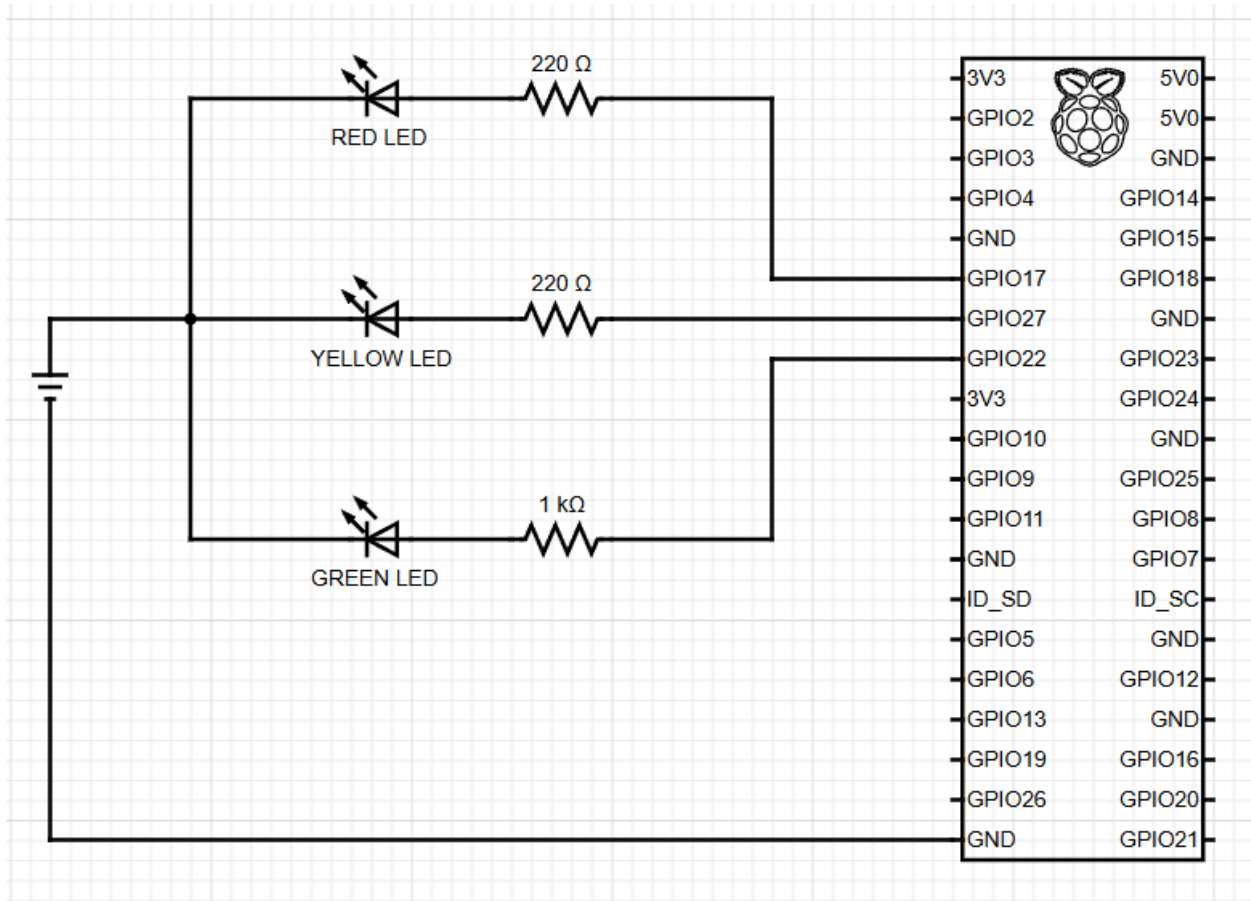
```
pi42@raspberrypi42:~/CSC615 $ echo 534 | sudo tee /sys/class/gpio/export
534
pi42@raspberrypi42:~/CSC615 $ ls /sys/class/gpio/
export  gpio534  gpiochip512  unexport
pi42@raspberrypi42:~/CSC615 $ ls /sys/class/gpio/gpio534
active_low  device  direction  edge  power  subsystem  uevent  value
pi42@raspberrypi42:~/CSC615 $ echo 534 | sudo tee /sys/class/gpio/unexport
534
pi42@raspberrypi42:~/CSC615 $ ls /sys/class/gpio/
export  gpiochip512  unexport
pi42@raspberrypi42:~/CSC615 $
```

And sure enough, I was able to write to the export and unexport file without any issues. So in my code, I made sure to correctly modify the pin number so that there would not be any errors.

Photo of completed circuit:



Hardware Diagram:



Analysis:

(If required for the assignment)

Screen shot of compilation:

```
pi42@raspberrypi42:~/CSC615 $ make clean
rm -f assignment1pigpio assignment1b
pi42@raspberrypi42:~/CSC615 $ make
gcc -Wall -pthread -o assignment1pigpio assignment1pigpio.c -lpigpio -lrt
gcc -o assignment1b assignment1b.c
pi42@raspberrypi42:~/CSC615 $
```

Screen shot(s) of the execution of the program:

```
pi42@raspberrypi42:~/CSC615 $ make clean
rm -f assignment1pigpio assignment1b
pi42@raspberrypi42:~/CSC615 $ make
gcc -Wall -pthread -o assignment1pigpio assignment1pigpio.c -lpigpio -lrt
gcc -o assignment1b assignment1b.c
pi42@raspberrypi42:~/CSC615 $ make run1
sudo ./assignment1pigpio
Controlling LEDs with PiGPIO
Initializing GPIO pins
Running...
Terminating.
pi42@raspberrypi42:~/CSC615 $ make run2
sudo ./assignment1b
Controlling LEDs with sysfs
Initializing GPIO pin 22
Initializing GPIO pin 27
Initializing GPIO pin 17
Running...
Unitializing GPIO pin 22
Unitializing GPIO pin 27
Unitializing GPIO pin 17
pi42@raspberrypi42:~/CSC615 $ make clean
```